Lecture 5 - Evolutionary Algorithms I

Created	@February 28, 2024 12:00 AM
Type	Lecture
Reviewed	☐

Evolutionary Algorithms Overview

- Evolutionary algorithms (EAs) are a subset of evolutionary computation, a generic population-based metaheuristic optimisation algorithm. They use mechanisms inspired by biological evolution, such as reproduction, mutation, recombination, and selection. The goal of EAs is to find optimal or near-optimal solutions to problems by iteratively improving a population of candidate solutions according to a defined fitness metric.
- Here's an overview of how evolutionary algorithms typically work:

1. **Initialisation**: Start with a randomly generated population of individuals. Each individual represents a potential solution to the problem and is typically encoded as a string of characters or numbers (known as chromosomes).
2. **Evaluation**: Each member of the population is evaluated based on a fitness function. The fitness function determines how well an individual solves the problem at hand.
3. **Selection**: Individuals are selected to contribute to the next generation based on their fitness. The selection process favours better-performing individuals, although it usually allows some less-fit individuals to maintain genetic diversity.
4. **Reproduction**: Selected individuals reproduce to create a new generation. Reproduction can involve various operators, such as:
    - **Crossover (or recombination)**: Two or more individuals combine to form one or more offspring, exchanging parts of their genetic material.
    - **Mutation**: Random changes are made to an individual's genetic material to introduce variability.
5. **Replacement**: The new generation of individuals replaces the old generation, and the algorithm returns to the evaluation step.
6. **Termination**: This iterative process continues until a termination condition is met, which could be a sufficient level of fitness, a maximum number of generations, or another specific criterion related to the problem.

- Types of evolutionary algorithms include:
    - **Genetic Algorithms (GAs)**: Use a representation that mimics biological chromosomes and apply crossover and mutation to evolve solutions.
    - **Evolution Strategies (ES)**: Emphasise the role of mutation and selection, often with self-adaptive mutation rates.
    - **Genetic Programming (GP)**: Evolve computer programs, typically represented as tree structures.

    - **Differential Evolution (DE)**: Focuses on the optimization of real-valued multi-modal functions and uses a specific type of mutation based on the differences between randomly selected pairs of individuals from the population.
- Evolutionary algorithms are used in various fields, including optimisation, engineering, economics, social sciences, and biology, for their ability to find good solutions in complex and multidimensional search spaces.
- Through evolution we observe desirable traits through genetic variation.
- Change or genetic variation comes from:
    - Mutations: changes in the DNA sequence,
    - Crossover: reshuffling of genes through sexual reproduction and migration between populations
    - Evolution is driven by natural selection – survival of the fittest
- Genetic variations that enhance survival and reproduction become and remain more common in successive generations of a population.

Key Terminology

- Evolutionary Algorithms → EAs **simulate** natural evolution (Darwinian Evolution) of **individual** structures at the genetic level using the idea of **survival of the fittest**, via processes of **selection**, **mutation** and **reproduction**(recombination).
- An **individual** (chromosome) represents a candidate solution for the problem.
    - e.g. $< 2\ 1\ 3\ 4 >$
- **Population** → a set of individuals currently "alive".
- **Population Evolution:**
    - The population is evolved from one **generation** (iteration) to another.
    - This is based on the **fitness** of individuals in a given environment, indicating how fit (close to the optimal solution) it is (i.e. the objective value).
        - e.g. $f(< 2\ 1\ 3\ 4 >)$
- **Hope** → the last generation will contain the final solution.
- **There are a set of evolutionary algorithms:**
    - **Genetic Algorithms**: Evolve binary strings (bit strings). Pioneered by Nils Aall Barricelli in 1954, and later by John Holland in 1975.
    - **Memetic Algorithms**: Introduced by Pablo Moscato in 1989. They combine local search optimisation with genetic algorithms.
    - **Evolutionary Programming**: Evolves parameters of a program with a fixed structure. Initiated by Lawrence J. Fogel, A. J. Owens, and M. J. Walsh in 1966.
    - **Evolution Strategies:** Use vectors of real numbers. Proposed by Ingo Rechenberg in 1973.
    - **Genetic Programming:** Evolves computer programs in tree form. Developed by John Koza in 1992.

- **Gene Expression Programming:** In this method, computer programs of different sizes are encoded in linear chromosomes of fixed length.
- **Grammatical Evolution:** Involves evolving solutions with respect to a specified grammar. Established by Michael O'Neill and Conor Ryan in 1998.

Genetic Algorithms

Pseudo Code for Genetic Algorithms

```
begin
    generate initial population; // initialise
    calculate fitness values; // evaluate population
    do
    {
        perform reproduction; // select parents
        recombine pairs with p_c; // apply crossover
        apply mutation with p_m; // mutate
        offspring/children
        calculate fitness values; // eval. population
        replace current population;
    } while termination criteria not satisfied;
end
```

1. **Generate initial population**: This is the first step where the algorithm creates a set of potential solutions randomly. Each solution in the population is called an individual or a chromosome.
2. **Calculate fitness values**: For each individual in the population, the algorithm evaluates how "fit" or how well it solves the problem at hand. The fitness function is problem-specific and determines how well a given solution is compared to others.
3. **Perform reproduction (select parents)**: This step involves selecting individuals from the current population to contribute to the next generation. The selection process often favors individuals with higher fitness, but it can include randomness to maintain genetic diversity.
4. **Recombine pairs with p_c**: Pairs of selected individuals are recombined or crossed over at a probability p_c. Crossover is a genetic operator used to vary the programming of a chromosome or chromosomes from one generation to the next. It is analogous to reproduction and biological crossover, upon which genetic algorithms are based.
5. **Apply mutation with p_m**: Mutation is another genetic operator that introduces random changes to individual parents to form children. This is done with a probability p_m to maintain diversity within the population and prevent premature convergence on suboptimal solutions.
6. **Offspring/children**: This refers to the new individuals generated after the crossover and mutation processes. They will form part of the next generation.

7. **Calculate fitness values**: Again, the algorithm evaluates the population after the genetic operators have been applied to generate offspring. This step assesses the newly formed individuals' fitness.
8. **Replace current population**: The algorithm replaces the older generation with the new generation of offspring. This can be done in various ways, like completely replacing the old generation, only replacing the less fit individuals, or using other replacement strategies.
9. **Termination criteria**: The algorithm continues the loop of selection, crossover, mutation, and replacement until a predefined termination condition is met. This condition could be a certain number of generations, a specific level of fitness that has been achieved, or other problem-specific criteria.
10. **End**: Once the termination condition is satisfied, the algorithm ends. The best solution found (i.e., the individual with the highest fitness) is typically returned as the output.

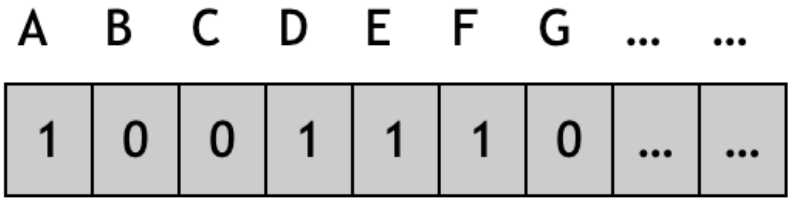
Components to a Genetic Algorithm

```
INITIALISE
loop while true
    EVALUATE
    if (TERMINATION CONDITION) break
    SELECT
    CROSSOVER
    MUTATE
    return BEST
end loop
```

- **Representation** → a genetic representation (encoding) for candidate solutions to the problem at hand.
- **Initialisation** → an initialisation scheme to generate the first population of candidate solutions.
- **Fitness (evaluation) function** → a fitness (evaluation) function that plays the role of the environment, rating solutions in terms of their fitness.
- **Selection** → a scheme for selecting mates (parents) for recombination.
- **Crossover** → crossover (recombination) exchanges genetic material between mates, producing offspring (children).
- **Mutation** → mutation perturbs an individual, creating a new one.
- **Replacement Strategy** → replacement strategy to select the surviving individuals for the next generation.
- **Termination Criteria** → various types of conditions that signal algorithm termination.
- **Parameter Values** → values for parameters such as population size and probabilities of applying genetic operators etc.

Components to a Genetic Algorithm → Representation

- **Representation** → a genetic representation (encoding) for candidate solutions to the problem at hand.
- GA's require representation of possible solutions as chromosome or genome-like objects.
- The most widely used representations are binary strings and lists, although other representations such as trees and multi-dimensional arrays are possible.
- A single chromosome or genome consists of separate elements or genes., each of which contains a value (or *allele*) which may be binary, integer, real-valued or even symbolic.
- A typical binary coded chromosome (the grey-shaded string below) would be of the form:



- Where A, B, C... represent separate elements of the problem and the values '0' and '1' the presence or absence of these elements.
- For example, Fischer and Leung (1998) used a 68-bit string to encode a multi-layer neural network spatial interaction model, in which individual bits were assigned meanings such as: bits 1,8 and 15 = presence/absence of hidden layers 1, 2 and 3 respectively; bits 2-7 = the number of nodes in the first hidden layer (hence<64); etc.

1. **Haploid structure**: In the context of GAs, this means that each individual in the population has a single set of chromosomes. In other words, each individual is represented by one chromosome.
2. **Fitness value**: Every individual (solution) in the population is evaluated to determine how well it solves the problem. This is quantified by the fitness value, which measures the quality of the solution. Higher fitness values indicate better solutions.
3. **Chromosome length**: Each chromosome has a fixed number of genes, which is the chromosome length. This number remains constant throughout the algorithm.
4. **Binary encoding**: Traditionally, each gene in the chromosome is represented as a binary value, either 0 or 1. This binary encoding is the allele value for that gene. Each gene's value represents a specific characteristic or parameter of the solution.
5. **Population size**: A GA operates on a population of individuals. The population size is fixed and defines the number of solutions that are simultaneously considered in each iteration of the algorithm.
6. **Generation**: Each iteration of the GA is referred to as a generation. In each generation, the algorithm performs selection, crossover, and mutation operations to create a new set of solutions (a new generation) from the current one.

Example

Problem: Maximise the function $f(x) = x^2$, where x is a binary-encoded integer.

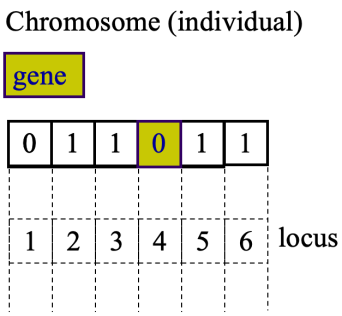
- Here's how we might represent the components of a GA for this problem:

1. **Chromosome Structure**: A chromosome could be a binary string where each bit represents a gene. For simplicity, let's assume a chromosome length of 4 bits.
2. **Initial Population**: Suppose we have a population size of 4 individuals. The initial population might look like this:
    - Individual 1: `1100` (binary for the integer 12)
    - Individual 2: `1010` (binary for the integer 10)
    - Individual 3: `0101` (binary for the integer 5)
    - Individual 4: `0011` (binary for the integer 3)
3. **Fitness Value Calculation**: We calculate the fitness value by plugging the binary-encoded integer into our function $f(x) = x^2$.
    - Fitness of Individual 1: $f(12) = 12^2 = 144$
    - Fitness of Individual 2: $f(10) = 10^2 = 100$
    - Fitness of Individual 3: $f(5) = 5^2 = 25$
    - Fitness of Individual 4: $f(3) = 3^2 = 9$
4. **Binary Encoding of Genes**: Each gene in the chromosome is already binary-encoded as either `0` or `1`. Each individual's chromosome represents a candidate solution to the problem.
5. **Fixed Population Size**: Our example maintains a population size of 4 throughout the GA process.
6. **Generation**: Each new set of individuals created after selection, crossover, and mutation forms a new generation. For instance, if after the operations the population changes to `1110`, `1101`, `1011`, and `0110`, that's considered one generation forward in the process.

- This is an abstract representation, and in a real-world scenario, the function to maximize and the length of the binary string would likely be more complex, potentially requiring a more sophisticated encoding scheme to represent a wider range of possible solutions.

Components to a Genetic Algorithm → Initialisation

- **Initialisation** → an initialisation scheme to generate the first population of candidate solutions.
- **Random Initialisation**: This is the process by which the initial population of potential solutions is generated. Each individual in the population (representing a possible solution) is created randomly to introduce genetic diversity.
- **Population Size**: This refers to the number of individuals in the initial population. The population size is a crucial parameter in genetic algorithms because it affects both the genetic diversity of the population and the computational resources required for the algorithm. It's typically set based on the complexity of the problem and the desired exploration of the solution space.



- **Genes and Alleles**: In the context of genetic algorithms, an individual's chromosome is analogous to a solution to the problem being solved. The chromosome is composed of multiple genes, each located at a specific position or locus on the chromosome. The value that a gene takes is called an allele. In binary-encoded GAs, an allele can be either 0 or 1.
- **Assigning Allele Values**: The slide describes the assignment of allele values to each gene on an individual's chromosome. The process is random, typically using a uniform random distribution where each allele has an equal chance of being 0 or 1. This is likened to flipping a coin; if the result of a random number generator is less than 0.5, the allele is set to 0; otherwise, it is set to 1.

Genes Alleles and Locus Further Explained

- **Chromosome [011011]**: A chromosome is a complete set of genetic material or a single entity within a population in the context of GAs. It represents a potential solution to the problem being tackled by the GA. The entirety of a chromosome can be likened to the recipe for a solution where each ingredient (gene) contributes to the overall outcome.
- **Gene[01]**: A gene is a specific part of a chromosome that encodes a particular piece of information or trait in the potential solution. In other words, each gene holds a piece of the puzzle that is a complete potential solution to the problem. In computational terms, a gene might represent a variable or a parameter that can influence the solution.
- **Allele[0/1]**: An allele is a variant form of a gene. Since GAs often use binary encoding, an allele can be either 0 or 1. Each allele corresponds to a different version or state of the trait that the gene represents. For example, if the gene represents a boolean condition in a solution, then allele `1` might represent true, and allele `0` might represent false.
- **Locus (plural: loci)[index 123456]**: This is the position or location of a gene on a chromosome. Each locus can be occupied by an allele. In a binary-encoded GA, this is simply the position of a bit within a binary string.

Components to a Genetic Algorithm → Fitness Calculation/Evaluation

- **Fitness (evaluation) function** → a fitness (evaluation) function that plays the role of the environment, rating solutions in terms of their fitness.
- Specification of a suitable fitness function is a key element in the design of a successful GA. It measures how good a single individual is in terms of meeting the problem objective
- We have to evaluate each one of the individuals in the population.

- The objective values are important by measuring the quality of the individuals: how close they are to the optimal solution or directly represent their quality.
- **Fitness value** indicates:
  - How fit the individual is to survive and reproduce under the current conditions
  - How much the current solution meets the requirements of the objective function
- **Fitness value** is obtained by applying the **fitness function** to the individual's chromosome
  - genotype (e.g. 101110) to phenotype (e.g. 1) by mapping

Fitness Calculation Further Explained:

- **Fitness Value**: It's a measure of how well an individual (a potential solution) performs with respect to the given problem. It reflects the individual's ability to compete in the environment defined by the problem's constraints and objectives.
- **Survival and Reproduction**: In the context of GAs, 'fit' individuals are those whose characteristics (solution attributes) are well-suited to the problem environment. These individuals are more likely to be selected for reproduction (i.e., to be used to create the next generation), as they have a higher chance of producing offspring with good solutions.
- **Objective Function Satisfaction**: The fitness value is determined by how much the solution represented by the individual meets the requirements of the objective function, which quantitatively measures how close a solution is to the optimum.
- **Genotype to Phenotype Mapping**: The individual's chromosome represents the genotype, which is the encoded form of the solution. The phenotype is the expressed form of the genotype, which in computational problems usually corresponds to the solution itself when decoded from the chromosome.

Example:

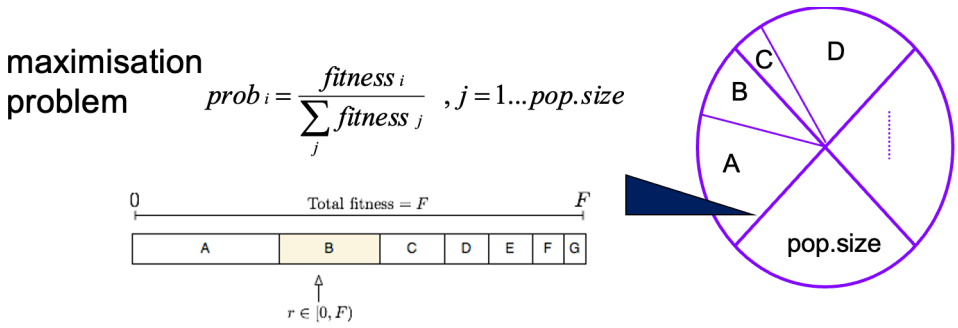
- Let's consider a simple GA designed to optimise the function f(x) = x^2, where x is an integer represented in binary form. The GA's objective is to maximize this function.
- **Genotype**: Suppose we have a chromosome (candidate solution) represented as `101110`. This is the genotype.
- **Phenotype**: When this binary sequence is translated into an integer, it becomes the phenotype. For example, `101110` in binary translates to 46 in decimal.
- **Fitness Function**: The fitness function here is f(x) = x^2. To calculate the fitness value for our example chromosome, we square the integer value of the phenotype:

  f(46) = 46^2 = 2116
- The fitness value for the chromosome `101110` would be 2116. This value would be used to compare against other individuals in the population to determine how 'fit' this individual is, relative to others. The higher the fitness value, the better the individual solution is at maximising the function $f(x) = x^2$.

Components to a Genetic Algorithm → Reproduction/Selection

- **Reproduction/Selection** → a scheme for selecting mates (parents) for recombination.

- **Reproduction in Genetic Algorithms**:
  - It involves selecting individuals from the current population to be parents for the next generation.
  - We are choosing parents to undergo crossover operations.
  - And we want to chose individuals with better fitness with a higher value/chance of being selected.
  - The selection is based on the fitness of the individuals, where fitter individuals have a higher chance of being selected – a concept known as **selection pressure.**
- **Selection Methods**:
  - **Roulette Wheel Selection**: Each individual is given a slice of a roulette wheel, the size of which is proportional to their fitness. A random spin of the wheel determines which individuals are selected.
  - **Tournament Selection**: A set number of individuals are chosen at random, and the fittest among them is selected.
  - **Rank Selection**: Individuals are ranked based on fitness, and selection is made based on this ranking.
  - **Truncation Selection**: Only the top percentage of individuals, ranked by fitness, are selected.
  - **Boltzmann Selection**: Selection chances are assigned based on a function of fitness and a temperature parameter that controls the distribution of selection probabilities.
- **Fitness Proportionate Selection (Roulette Wheel Selection)**:
  - Each individual's probability of selection is calculated as the ratio of its fitness to the total fitness of the population.
  - While fitter individuals are more likely to be selected, there is still a chance for less fit individuals to be chosen.
  - The expected number of offspring of each individual is proportional to its fitness, particularly for problems where the goal is to maximise the objective function.
- **Calculating Selection Probability**:
  - For a maximisation problem, the probability of an individual i being selected is calculated as $prob_i = \frac{fitness_i}{\sum_{j=1}^{pop.size} fitness_j}$, where $fitness_i$ is the fitness of individual i, and the denominator is the sum of fitnesses of all individuals in the population.
- **Visualisation of Roulette Wheel**:



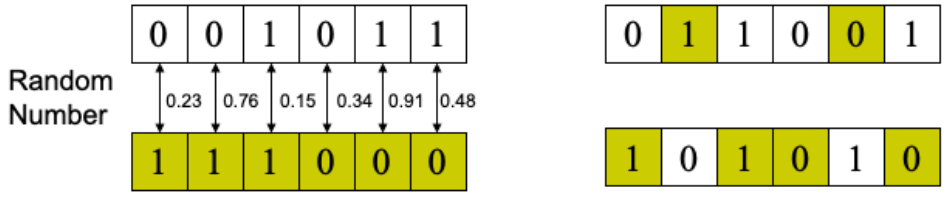
- A diagram might be used to illustrate the roulette wheel method, with each individual's section size representing their selection probability.
- By applying these methods, genetic algorithms simulate the process of natural selection, where the fittest individuals are given more opportunities to pass on their genes to subsequent generations, thereby guiding the population towards better solutions over time.

Components to a Genetic Algorithm → Crossover

- **Crossover** → crossover (recombination) exchanges genetic material between mates, producing offspring *(children).*
- Selected pairs/mates (parents) are recombined to form new individuals (children) — exchange of genetic material
- Crossover is applied with a **crossover probability** p_c (crossover rate) which, in general, is chosen close to 1.0
  - Large $\geq 70\% \ (0.7)$
  - Controls the probability of applying crossover (or else solutions pass through to mutation step)
- **Crossover Operators**
  - **One-Point Crossover (1PTX)**:
    - A single crossover point on both parent organisms' chromosome is selected.
    - All data beyond that point in the chromosome string from one parent is swapped with the data from the other parent.
    - The resulting chromosomes are a mixture of both parents, providing genetic diversity.
    - **One Point Crossover (1PTX) General outline:**
    - Generate a random number in [0,1]
    - If it is **smaller than a crossover probability** p_c then:
      - Select a random crossover site in $[1, \text{chromosome length}]$
      - Split individuals at the selected site
      - Exchange segments between pairs to form two new individuals
    - Else,
      - Copy the individuals as new individuals
  - **Two-Point Crossover (2PTX)**:
    - Instead of one point, two points are chosen on the parent organisms' chromosome.
    - Everything between the two points is swapped between the parent organisms, providing even more diversity than one-point crossover.
  - **K-Point Crossover:**
    - K-Point Crossover is a generalisation of the one-point and two-point crossover methods.

- In this crossover technique, you don't just choose one or two points on the chromosome to perform the crossover; instead, you select 'K' points, where 'K' can be any positive integer. This means that the chromosomes are divided into 'K+1' segments.

- **Uniform Crossover (UX)**:



  - Instead of swapping contiguous segments of genes, the uniform crossover considers each gene independently.
  - **uniform crossover** considers each bit in the parent strings for exchange with a probability of <0.5 then they are swapped
  - For each gene, a random number is generated and, depending on this number, either the mother's or father's gene is passed on to the offspring.
  - Typically, there's a 50% chance for each parent to pass on any given gene to the offspring.

Components to a Genetic Algorithm → Mutation

- **Mutation** → mutation *perturbs an individual, creating a new one.*
- **Any offspring** can be exposed to mutation.
- Loop through all the alleles of all the individuals, one by one.
  - If that allele is selected for mutation with a given **probability** p_m (mutation rate), we can either **change it** by a small amount or **replace it** with a new value.
  - For **binary representation**, mutation corresponds to a **bit flip**.
- Mutation provides **diversity** and allows GAs to **explore** different regions of the search space (escaping).
- **Mutation rate** is typically chosen to be very small (0.001-0.01)
  - Choosing p_m as $\frac{1}{\text{chromosome length}}$ implies that, on average, a single gene will be mutated for an individual.

Further explained:

- Mutation is a mechanism within genetic algorithms (GAs) that introduces variations by altering one or more gene values in the chromosomes of the offspring. It serves as a genetic operator that changes the individual chromosomes, thus providing genetic diversity and allowing the population to explore a broader range of the search space. This exploration is crucial for avoiding local optima and ensuring that the GA does not prematurely converge on a suboptimal solution.
- **Benefits of Mutation:**

    - Introduces new genetic structures in the population by modifying existing chromosomes.
    - Increases diversity within the population, which can be vital if the initial population does not represent the full breadth of the solution space.
    - Helps prevent premature convergence on suboptimal solutions by maintaining a diverse pool of genetic material.
- **Drawbacks of Mutation:**
    - **If the mutation rate is too high, it can disrupt convergence and lead to a random walk in the search space, which is inefficient.**
    - **Excessive mutation can destroy good solutions by introducing too many random alterations.**
    - **Too frequent mutations can lead to a loss of important genetic information that might have been beneficial for finding the optimal solution.**
- **Example of Mutation:**
    - Let's consider a binary-encoded chromosome from a GA that represents a candidate solution, and assume the mutation rate p_m is set to a standard 0.01 (1%).
    - Given a chromosome:
    - Original Chromosome: **101100**
    - During the mutation phase, we would examine each bit and decide whether to mutate it based on the mutation rate. Let's generate a random number for each bit and see if it falls below the mutation rate:
    - Random numbers for each bit: [0.02, 0.95, 0.10, 0.87, 0.03, 0.99]
    - Comparing each random number with the mutation rate, we mutate the bits where the random number is less than p_m:
    - Since the random numbers for the 1st and 5th bits are less than 0.01, we flip these bits.
    - The new chromosome after mutation would be:
    - Mutated Chromosome: 001101
    - Here, the 1st bit has changed from '1' to '0' and the 5th bit from '0' to '1'. The rest of the bits remain the same. This new chromosome will now carry a slightly different set of traits into the next generation, thereby contributing to the genetic diversity of the population.

Components to a Genetic Algorithm → Replacement Strategy

- **Replacement Strategy** → replacement strategy to select the surviving individuals for the next generation.
- In genetic algorithms (GAs), the replacement strategy is a method used to select individuals from the current population to be carried forward to the next generation.
- The strategy determines how a new population (offspring) replaces an older one (parents).

Replacement Strategy:

- **Variety of Strategies**: Multiple strategies exist for determining how the new offspring replace members of the current population.
- **Generation Gap (α)**:
  - **Defined as $\alpha \in [1/N, 1.0]$, where N is the population size.**
  - Controls the fraction of the population to be replaced each generation.
  - **The number of offspring produced each generation is given by $g = \alpha \times N$.**
- **(Trans-)Generational GA → parents and offspring form offspring**
  - A type of genetic algorithm where a significant portion, often the entirety, of a population is replaced with offspring in each generation, promoting substantial genetic diversity.
  - **This is when $g > 2$, which means $\alpha > 2/N$.**
  - N individuals produce αN offspring, so $(N + \alpha N) \rightarrow N$
  - αN replaces worst αN of N
    - largest generation gap where $\alpha = 1.0$ yields $g = N$
    - GA relies on improvement of average objective values from one population to another
      - always a good idea not to lose the best solution found so far
  - **Elitism** → A survival of the fittest approach whereby only the fittest individuals are carried onto the next generation. Elitism protects the fittest individuals in any given generation from crossover and mutation so they survive to the next generation without change.
  - Sort $N + \alpha N$ and choose N best (**elitism**).
  - A large generation gap $where(\alpha = 1)$ results in all individuals being replaced each generation (full generational replacement).
  - **The goal is to improve average fitness values from one generation to the next.**
  - It's important to preserve the best solutions found (elitism) by sorting the combined population of parents and offspring and selecting the top N individuals for the next generation.
- **Steady-State GA → offspring are only ones in offspring based on number**
  - A genetic algorithm approach where only a few members of the population are replaced at a time, leading to a more gradual evolution and maintaining a more constant population size.
  - **Typically has $g = 2$, which means $\alpha = 2/N$.**
  - Only two offspring are produced at a time and they replace individuals in the existing population using different methods:
    - **Method 1**: Two offspring replace their two parents.
    - **Method 2**: Two offspring replace the worst two individuals in the population.
    - **Method 3**: The best two individuals out of the parents and offspring replace two parents (**elitism**).
    - **Method 4**: The best two individuals out of the parents and offspring replace the worst two individuals in the population (**strong elitism**).

▼ **Further explanation including constant population size**

Elitism and Strong Elitism in Genetic and Memetic Algorithms

Elitism and **strong elitism** are strategies used in genetic and memetic algorithms to ensure that the best individuals (solutions) are preserved across generations. Both strategies typically maintain the population size constant. Here's how they work and their effects on population size:

Elitism

Elitism ensures that a certain number of the best individuals from the current generation are carried over to the next generation without alteration. This strategy helps maintain high-quality solutions and improves convergence rates.

Process:

1. **Selection of Elites:** The top k individuals (elites) based on fitness are selected from the current population.
2. **Crossover and Mutation:** The rest of the new population is created using crossover and mutation on selected individuals (excluding elites).
3. **Replacement:** The k elites replace the worst k individuals in the new population, ensuring the best individuals are preserved.

Effect on Population Size:

- The population size remains constant because the number of elites is subtracted from the number of individuals created through crossover and mutation.

Strong Elitism

Strong elitism is a stricter form of elitism where the best individuals are not only preserved but are also used more aggressively in the creation of new solutions.

Process:

1. **Selection of Elites:** The best k individuals are selected from the current population.
2. **Crossover and Mutation:** These elites can be given a higher probability of being selected for crossover and mutation, or they may be cloned directly into the next generation.
3. **Replacement:** Similar to standard elitism, the best k individuals replace the worst k individuals in the new population.

Effect on Population Size:

- The population size remains constant because the total number of individuals (including both elites and those produced by genetic operators) remains the same as the initial population size.

Example

Let's consider an example with a population size of 10 and 2 elites:

1. **Initial Population:** 10 individuals.
2. **Selection of Elites:** The top 2 individuals are selected as elites.

3. **Crossover and Mutation:** 8 new individuals are created from the remaining population (excluding the elites).
4. **Replacement:** The 2 elites are included in the new population, replacing the 2 worst individuals out of the 8 newly created ones.
5. **Next Generation:** 10 individuals (8 new + 2 elites).

Conclusion

Both elitism and strong elitism maintain a constant population size in genetic and memetic algorithms. They ensure that the best individuals are preserved and potentially used more aggressively in generating new solutions, but the overall number of individuals in the population remains unchanged across generations.

Example:

- Let's delve into the details and mathematics for each replacement method within a Steady-State Genetic Algorithm (GA) where the population size is N and the generation gap is $\alpha = 2/N$. For our example, let's assume $N = 100$, which means $\alpha = 0.02$ or 2%

Steady-State GA Replacement Methods:

1. **Method 1: Two offspring replace two parents**
   - No calculation for fitness is needed in this method.
   - Offspring A and B are produced through crossover and mutation from parents 1 and 2.
   - Offspring A and B directly replace parents 1 and 2, irrespective of their fitness scores.
   - In essence, the parents are simply updated to the new offspring.
2. **Method 2: Two offspring replace the worst two of the population**
   - Calculate the fitness of all individuals in the population.
   - Identify the two individuals with the lowest fitness, say individuals 99 and 100 with fitness scores of 0.1 and 0.2, respectively.
   - Offspring A and B are produced and their fitness scores are calculated, say 0.5 and 0.6, respectively.
   - The two offspring replace the two least fit individuals, improving the overall fitness of the population.
   - Mathematically: if $f_{offspring} > f_{worst}$, then replace.
3. **Method 3: Best two of (parents and offspring) replace two parents (elitism)**
   - Fitness scores for parents (1 and 2) and offspring (A and B) are calculated.
   - Let's assume parents have fitness scores of 0.7 and 0.8, while offspring have scores of 0.5 and 0.6.
   - Because we are using elitism, we will choose the best between the parents and offspring.
   - In this case, parents are better, so no replacement occurs, maintaining the highest fitness individuals.

4. **Method 4: Best two of (parents and offspring) replace the worst two of the population (strong elitism)**
    - Calculate fitness scores for all individuals and offspring.
    - Let's say the parents have fitness scores of 0.7 and 0.8, offspring have scores of 0.85 and 0.9, and the two worst individuals in the population have scores of 0.1 and 0.2.
    - Compare the fitness of the parents and offspring and select the best two.
    - Offspring are better with scores of 0.85 and 0.9, so they replace the two worst individuals in the population.
    - The population's average fitness is now higher due to the replacement of lower-fitness individuals with higher-fitness offspring.

Example Calculations:
In method 2, if the total fitness of the population is F_{total} before replacement and f_{99} and f_{100} are the fitness scores of the worst individuals, and f_A and f_B are the fitness scores of the offspring:

- Initial Average Fitness = F_{total}/N
- New Total Fitness after Replacement = $F_{total} - f_{99} - f_{100} + f_A + f_B$
- New Average Fitness = $(F_{total} - f_{99} - f_{100} + f_A + f_B)/N$
- This way, you can quantitatively evaluate the impact of the replacement methods on the population's fitness.

Components to a Genetic Algorithm → Termination Criteria

- Termination criteria in genetic algorithms are conditions set to determine when the evolutionary process (main loop) should stop. The criteria ensure that the algorithm doesn't run indefinitely and include the following:
- **Maximum Number of Generations**: The algorithm stops after a predefined number of generations have been produced. This prevents the process from running too long and consuming excessive computational resources.
- **Fitness Goal Achievement**: The algorithm terminates if a candidate solution reaches or exceeds a desired level of fitness, which signifies that an acceptable solution has been found.
- **Population Convergence**: If the population of candidate solutions becomes too similar or converges, indicating a lack of diversity and that further iterations are unlikely to produce better solutions, the algorithm stops.
- **Stagnation**: The process ends if there's no significant improvement in the best fitness value for a predetermined number of generations, suggesting that the algorithm is stuck in a local optimum.
- **Combination of Conditions**: Often, a combination of these criteria is used for more robust control over the termination of the algorithm, allowing it to conclude once an optimal or satisfactory solution is reached or when further search is deemed unproductive.

Convergence

- **Convergence** is where successive offspring produced by the genetic algorithm are homogenous. These are the maxima. This produces a solution to the algorithm.
- **Premature convergence** occurs when the population converges too quickly to an individual of **poor fitness and objective value**. This is bad
- Convergence is defined as the **progression towards uniformity** (individuals become alike).
  - **Gene convergence:** a location on a chromosome is converged when 95% of the individuals have the same gene value for that location.
  - **Population (Genotypic) convergence:** a population is converged when all the genes have converged (all individuals are alike – they might have different fitness).
  - **Phenotypic Convergence:** average fitness of the population approaches to the best individual in the population (all individuals have the same fitness).

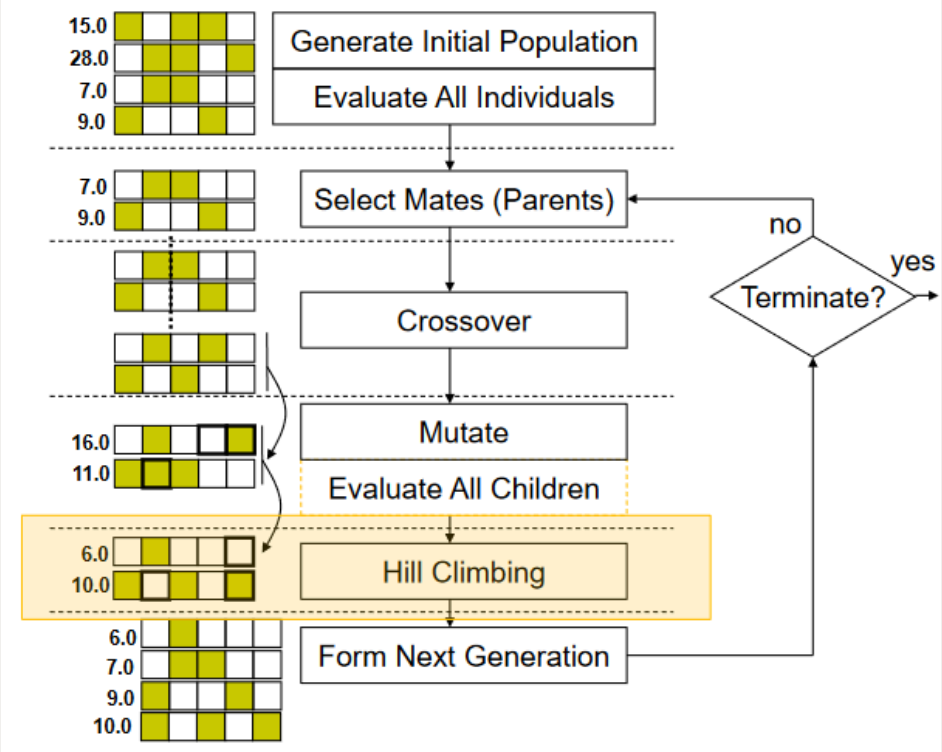
▼ Key Features of EAs

- Population based search approaches.
  - Be independent of initial starting point(s).
    - Start the search from many points in the search space.
  - Conduct search in parallel over the search space.
    - Implicit parallelism.
- Avoid converging to local optima.
- Balances exploration and exploitation?
- This may be used together with other approaches (hybrids).

Memetic Algorithms (MAs)

- **Meme** → a **contagious** piece of **information**. - Richard Dawkins
- Memes are similar to **local** refinement/ local search.
- Gene vs. Meme: **memes can change, evolve using rules and time scales**, unlike the traditional genetic ones.
- MAs aim to improve GAs by embedding local search
  - they make use of the **exploration** capabilities **of GAs**;
  - and the **exploitation** capabilities **of local search**.
  - They therefore have an explicit algorithm to balance exploitation and exploration.
- **Result:** MAs are much **faster** and more **accurate** than GAs on some problems.
  - “state of the art” for many problems

▼ Generic Memetic Algorithm



The diagram in the photo depicts the flow of a Memetic Algorithm (MA), which is an evolutionary computing approach that combines the global search capability of a Genetic Algorithm (GA) with local search techniques for refinement:

1. **Generate Initial Population**: The algorithm starts by creating a diverse set of potential solutions.
2. **Evaluate All Individuals**: Each member of the population is assessed to determine how well it solves the problem (fitness).
3. **Select Mates (Parents)**: Based on their fitness, some individuals are chosen to be parents for the next generation.
4. **Crossover**: Pairs of parents are combined or "crossed over" to produce offspring with mixed characteristics.
5. **Mutate**: The offspring undergo random mutations to introduce additional variability.
6. **Evaluate All Children**: The fitness of all the new offspring is evaluated.
7. **Hill Climbing**: This is a local search process where individual offspring are incrementally altered to improve their fitness. It's like looking for the highest point by taking steps uphill.
8. **Form Next Generation**: The algorithm decides which individuals make it to the next generation. This could involve a mixture of the best parents and children or just the best individuals regardless of their parentage.
9. **Terminate?**: The algorithm checks if it should stop, which could depend on several factors like a fixed number of generations, a satisfactory solution has been found, or no improvement in solutions over time.

The process loops from step 3 to step 8 until the termination criteria are met. Memetic Algorithms leverage the exploration power of genetic algorithms for a broad search and exploitation power of local search (like hill climbing) to intensively search local regions. This dual approach often leads to faster and more accurate convergence on optimal or near-optimal solutions compared to using genetic algorithms alone.

```
CreateInitialSolution(); // create initial population of solutions
repeat
        SelectParents(); // select solutions from the population to breed
        Crossover(); // apply crossover operator with a given probability
        Mutate(); // apply mutation opr=erator with a given probability
        LocalSearch();
        Evaluate();
        ReplaceSolutions(); // generate new population of solutions
until TerminationCriteriaSatisfied();
```

Where to put the hill climbing

Integrating hill climbing into a memetic algorithm at various stages significantly influences the algorithm's dynamics and its ability to find optimal solutions. Let's consider the effects of inserting hill climbing at different points, including between crossover and mutation, within the memetic algorithm framework.

Before Selection

Applying hill climbing before selection can enhance the population's average fitness. This pre-selection improvement can potentially lead to faster convergence by ensuring that only the fittest individuals are chosen for breeding. However, this approach comes with certain risks:

- **Premature Convergence:** Enhancing every individual before selection might lead to a uniform population too early, diminishing genetic diversity and potentially causing premature convergence to suboptimal solutions.
- **Computational Overhead:** Executing hill climbing on the entire population before selection increases computational demands, which may not be justifiable, especially if the improvements are marginal for already low-fitness individuals.

After Selection and Before Crossover

Once selection narrows down promising candidates, applying hill climbing can further refine these individuals. This targeted improvement means computational resources are spent on individuals with a higher likelihood of contributing to optimal offspring. However, considerations include:

- **Reduced Diversity:** This stage still risks reducing diversity, as hill climbing may narrow the range of solutions, potentially making the population too similar and limiting the genetic algorithm's exploratory capabilities.
- **Efficiency vs. Exploration:** It's crucial to balance the depth of hill climbing to ensure that while selected individuals are optimised, there remains sufficient variability for effective crossover and mutation.

Between Crossover and Mutation

Inserting hill climbing between crossover and mutation is a strategic choice that leverages the genetic diversity introduced through crossover before refining these solutions. This timing can be particularly effective because it:

- **Refines New Combinations:** By applying hill climbing to the novel genetic combinations produced during crossover, it enhances the quality of these solutions before they are further altered by mutation. This can lead to a more refined exploration of the solution space.
- **Balances Exploration and Exploitation:** This intermediate step maintains the diversity introduced by crossover while ensuring that the solutions are as optimized as possible before undergoing mutation, which introduces additional variability and innovation into the population.

After Mutation

Applying hill climbing after mutation takes advantage of the full spectrum of genetic operations, using hill climbing as a final refinement step. This approach allows for:

- **High-Quality Solutions:** The post-mutation application of hill climbing can significantly improve the fitness of solutions by fine-tuning the diverse genetic material produced by crossover and mutation.
- **Maintained Diversity:** Since hill climbing is applied after introducing new genetic material, it doesn't compromise the diversity needed for effective exploration of the solution space.

Adaptive Use

An adaptive approach to applying hill climbing—modifying its intensity and timing based on current search conditions—can dynamically balance exploration and exploitation, potentially leading to more efficient and effective search processes. This strategy adapts the use of hill climbing to the evolving needs of the algorithm, optimizing its impact on solution quality and diversity.

▼ Example Design

Designing a Genetic/Memetic Algorithm for MAX-SAT:

Representation	bit string of length N (truth assignment for each variable)
Initialisation	randomly generate initial population, population size = N
Fitness Function	(# of clauses - C); # of unsatisfied clauses
Mate Selection	tournament selection with a tour size of 2
Crossover	1PTX, crossover probability = 0.99
Mutation	random bit-flip, mutation probability $\frac{1}{N}$ $\therefore$ hill climbing: Davis's Bit Hill Climbing
Replacement	Steady State GA, best two (of parents and offspring) replaces the worst two individuals (strong elitism)

MAX-SAT Walkthrough

Initialisation

- **Problem Instance :** $(a \vee b) \wedge (\neg d \vee f) \wedge (\neg a \vee c) \wedge (b \vee \neg f) \wedge (\neg b \vee c) \wedge (c \vee e)$
- **Goal:** Assign truth values to variables to satisfy the maximum number of clauses.
- Clauses: Combinations of literals (variables or their negations) connected by logical OR (disjunction) and AND (conjunction) operations.
- **Population and Chromosomes**:
  - **Population size is set to 4** - meaning there are four unique potential solutions to start with.
  - **Chromosome size is 6** - corresponding to the six literals present in the MAX-SAT problem (a, b, c, d, e, f).
- **Initialisation Process**:
  - Create four initial individuals (solutions), each with a chromosome consisting of 6 genes.
  - Chromosomes are binary strings, where each gene represents the truth value of a literal.
  - Genes are initialised by generating random numbers between 0 and 1 for each gene. **If a random number is less than 0.5, the gene is assigned the allele 0; if 0.5 or greater, the gene is assigned the allele 1.**
- **Example Chromosome Creation**:
  - For example, individual 1 might have a chromosome initialised with the binary string `101110`, which is derived from random numbers such as <0.98, 0.85, 0.13, 0.04, 0.47, 0.44>.
  - The binary string `101110` represents the initial assignment of truth values to the literals a, b, c, d, e, f. For instance, `1` might mean 'true' and `0` 'false', for the literals in each clause of the MAX-SAT problem.
- **Example Chromosomes**:
  - Individual 1: <0.98, 0.85, 0.13, 0.04, 0.47, 0.44> → `110000`
  - Individual 2: <0.09, 0.63, 0.22, 0.54, 0.07, 0.37> → `010100`
  - Individual 3: <0.21, 0.03, 0.72, 0.84, 0.19, 0.49> → `001100`
  - Individual 4: <0.61, 0.43, 0.87, 0.53, 0.97, 0.14> → `101110`
- The binary strings correspond to the initial truth value assignments for literals a-f. These are used as the starting point for the genetic algorithm's evolutionary search for an optimal or satisfactory solution to the MAX-SAT problem.

Fitness Calculation

- **Chromosomes and Genotypes**: In the GA context, each individual solution is represented as a chromosome. A chromosome consists of genes, where each gene represents a variable (or literal) in the problem. The specific arrangement of genes within a chromosome is referred to as the genotype. For example, a chromosome represented as `101110` is a genotype that encodes a solution to the problem.

- **Phenotype**: The phenotype is the expressed behavior or trait that results from the genotype. In the context of the MAX-SAT, the phenotype would be the actual set of truth values assigned to the variables based on the genotype's encoding. For instance, a genotype of `101110` might correspond to a phenotype where the first, third, fourth, and sixth variables are true, and the second and fifth are false.
- **Fitness Function**: The fitness function evaluates how many clauses are satisfied by the phenotype. A higher fitness value indicates a better solution, as more clauses are satisfied. Typically, the fitness value is the count of satisfied clauses.

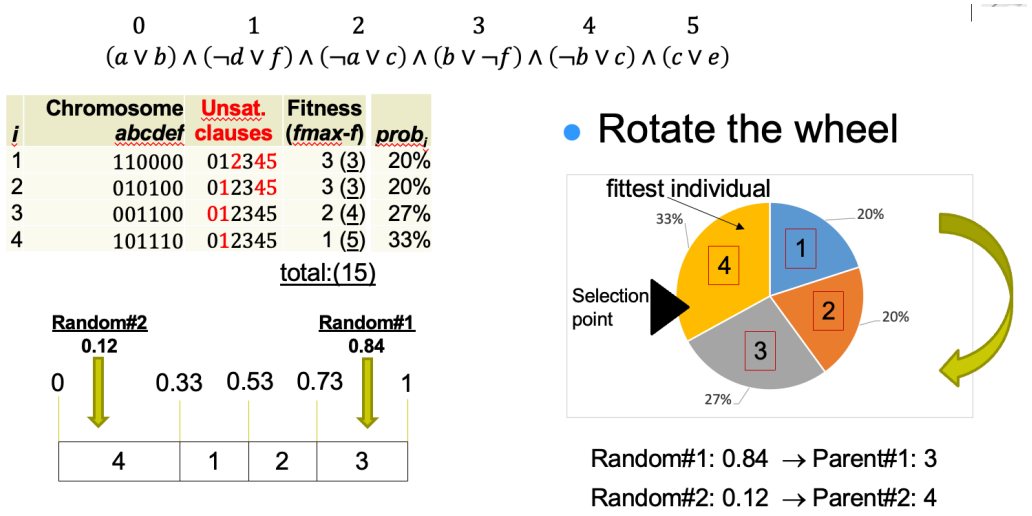
$$\overset{0}{(a \vee b)} \wedge \overset{1}{(\neg d \vee f)} \wedge \overset{2}{(\neg a \vee c)} \wedge \overset{3}{(b \vee \neg f)} \wedge \overset{4}{(\neg b \vee c)} \wedge \overset{5}{(c \vee e)}$$

i	chromosome *abcdef*	unsatisfied clauses	fitness
1	110000	012345	3
2	010100	012345	3
3	001100	012345	2
4	101110	012345	1

Using the provided example:

1. **Individual 1 (Genotype `110000`)**:
   - **Phenotype**: Variables a and b are true; the rest are false.
   - **Unsatisfied Clauses**: 2,4,5.
   - **Fitness**: 3 clauses are unsatisfied with this assignment.
2. **Individual 2 (Genotype `010100`)**:
   - **Phenotype**: Variable b is true, d is true; the rest are false.
   - **Unsatisfied Clauses**: 1,4,5.
   - **Fitness**: 3 clauses are unsatisfied.
3. **Individual 3 (Genotype `001100`)**:
   - **Phenotype**: Variables c and d are true; the rest are false.
   - **Unsatisfied Clauses**: 01
   - **Fitness**: 2 clauses are unsatisfied with this assignment.
4. **Individual 4 (Genotype `101110`)**:
   - **Phenotype**: Variables a, b, e, and f are true; c is false, d is true.
   - **Unsatisfied Clauses**: 1.
   - **Fitness**: Only 1 clause is unsatisfied with this assignment.

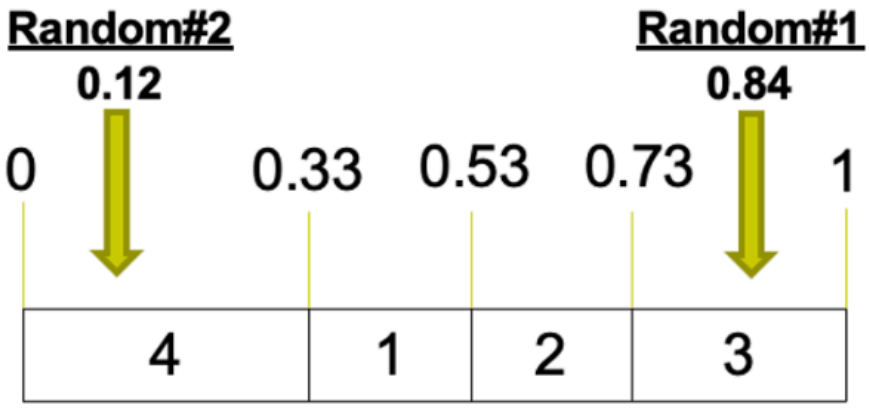
Roulette Wheel Selection



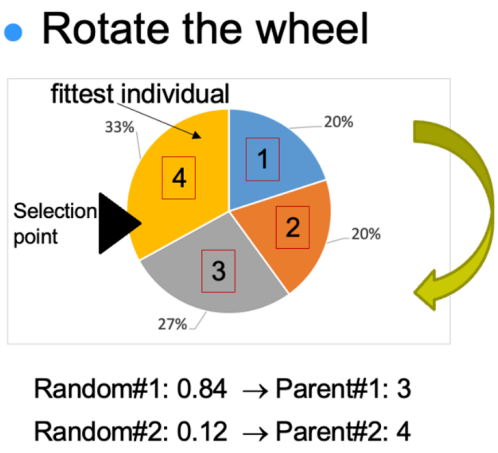
- **Roulette Wheel Selection for MAX-SAT Problem**:
  - This method is used to probabilistically select chromosomes from the population for reproduction based on their fitness.
- **MAX-SAT Chromosomes and Fitness**:
  - Each chromosome represents a possible solution to the MAX-SAT problem, with genes representing truth values for the literals.
  - Fitness is calculated based on how many clauses are satisfied by the solution that the chromosome represents.
  - The slide shows 4 chromosomes, each with an associated fitness score representing the number of clauses satisfied.
- **Probability Assignment**:
  - The probability of selection for each chromosome is calculated relative to the total fitness of the population.
  - To find the selection probability for a chromosome, divide the fitness of the chromosome by the total fitness of all chromosomes.
- **Mathematical Calculation of Probabilities**:
  - Suppose we have a total fitness score for the population (sum of fitness scores of all individuals), which is 15.
  - Recall the formula for the **Maximisation Problem** $prob_i = \frac{fitness_i}{\sum_{j=1}^{pop.size} fitness_j}$ **we need the number of satsified clauses for the fitness here**
  - We need to get the number of satisfied clauses $Fitness = f_{max} - f$
  - Chromosome 1 has a fitness of 3 (6 total variables - 3 unsatisfied clauses = 3 satisfied clauses). Its selection probability is $\frac{3}{15} = 20\%$.
  - Chromosome 2 also has a fitness of 3, so its selection probability is the same, $\frac{3}{15} = 20\%$

- Chromosome 3 has a fitness of 4 (6-2), giving it a selection probability of $\frac{4}{15} \approx 27\%$
- Chromosome 4 has a fitness of 5 (6-1), making its selection probability $\frac{5}{15} \approx 33\%$

- **Performing Selection**:
  - A roulette wheel is conceptualised, where each segment's size is proportional to the chromosome's selection probability.
  - A random number between 0 and 1 is generated to simulate spinning the roulette wheel.
  - If the random number falls within the segment of a particular chromosome, that chromosome is selected.
- **Example Selection**:



  - Two random numbers are generated: 0.84 and 0.12.
  - For Random#1 (0.84), it corresponds to Parent#1, which is the 3rd Chromosome.
  - For Random#2 (0.12), it corresponds to Parent#2, which is 4th Chromosome.
- **Visualisation and Outcome**:



- The diagram may show a wheel with four segments of different sizes based on the probabilities. The selection point indicates where the 'spin' lands.
- The outcome is that Chromosome 3 and Chromosome 4 are selected as parents for the next generation's crossover operation.

Tournament Selection

Tournament Selection

- Run a number of "**tournaments**" among **randomly** chosen individuals (of tour size) selecting the one with the best fitness at the end.
  - This is then repeated for selecting each parent to be recombined
- **Process**
  - Select n randomly chosen distinct parents from the population, where n is the tournament size.
  - From these n parents, compare and sort in ascending/descending order based on fitness values.
  - The parent with the highest fitness value will be added to the mating pool.
- **Example: MAX-SAT Tournament Selection**

$$\overset{0}{(a \vee b)} \wedge \overset{1}{(\neg d \vee f)} \wedge \overset{2}{(\neg a \vee c)} \wedge \overset{3}{(b \vee \neg f)} \wedge \overset{4}{(\neg b \vee c)} \wedge \overset{5}{(c \vee e)}$$

i	chromosome abcdef	unsatisfied clauses	fitness
1	110000	012345	3
2	010100	012345	3
3	001100	012345	2
4	101110	012345	1

Tour Size = 3, 1st Parent:

- Throw a random number between 1 and 4 (population size) for three times:
  - [3, 1, 2] → represent individuals i
- Tournament selection chooses 3 individuals at random, then individual #3 with the fitness of 2 is returned as the first parent.

Tour Size = 3, 2nd Parent:

- Throw a random number between 1 and 4 (population size) for three times:
  - [4, 1, 3]
- Tournament selection chooses 3 individuals at random, then individual #4 with the fitness of 1 is returned as the second parent.

▼ **Tour size further explained**

In tournament selection, the number of individuals that win a tournament depends on the number of tournaments conducted and the tournament size, not just a fixed number of winners. Here's a breakdown of how tournament selection works and how the number of winners is determined:

Tournament Selection Process

1. **Tournament Size:** This is the number of individuals that compete in each tournament. Let's denote this as T.
2. **Number of Tournaments:** The number of tournaments conducted usually equals the number of individuals required for crossover in the next generation. Let's denote the population size as N. If we need to create N offspring, we will typically conduct N tournaments.

Steps in Tournament Selection:

1. **Select T Individuals:** Randomly choose T individuals from the population.
2. **Select Winner:** The individual with the best fitness among the T selected individuals is the winner of the tournament.
3. **Repeat:** Repeat the above steps until the desired number of individuals (typically N) are selected.

Example:

Let's consider an example with a population size $N = 10$ and a tournament size $T = 3$:

1. **Conduct 10 Tournaments:** Since we need 10 individuals to create the next generation, we conduct 10 tournaments.
2. **Select Winners:** Each tournament selects the best individual among the 3 randomly chosen participants.

Number of Winners:

- **Unique Winners:** The number of unique winners can vary. It's possible for the same individual to win multiple tournaments, especially if it has significantly higher fitness compared to others. This means that the total number of winners is not necessarily equal to the population size.
- **Exact Count:** The exact number of unique winners depends on the distribution of fitness in the population and the tournament size. In practice, individuals with higher fitness have a higher probability of winning multiple tournaments, which can lead to fewer unique winners.

Key Points:

1. **Tournament Size T:** Larger tournament sizes increase the selection pressure, meaning stronger individuals are more likely to win multiple tournaments.
2. **Number of Tournaments:** The number of tournaments usually matches the number of individuals required for the next generation (often equal to the population size N).
3. **Probability of Winning:** Individuals with higher fitness values have a higher probability of winning in multiple tournaments.

Conclusion:

- **Number of Winners:** The total number of individuals that win in tournament selection is equal to the number of tournaments conducted, typically matching the population size N. However, the

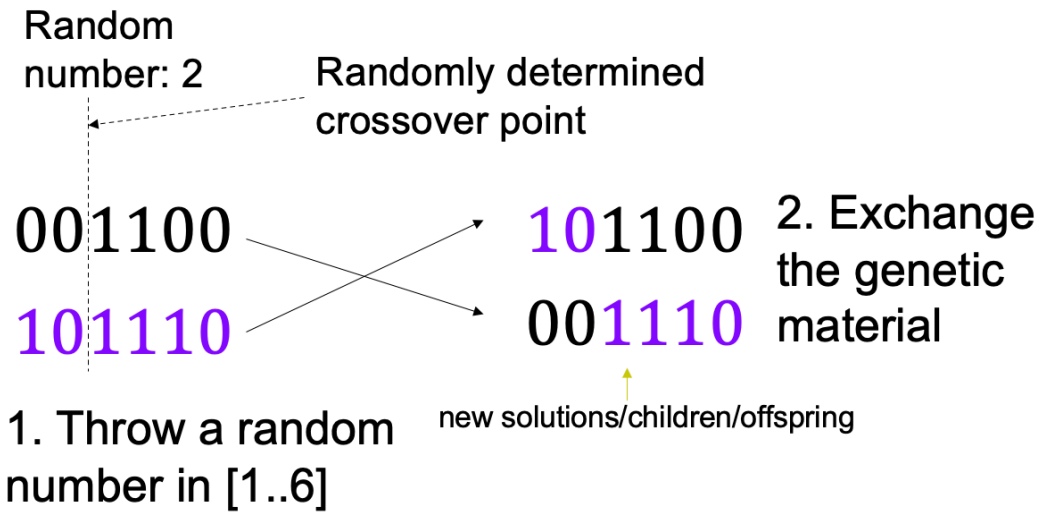
number of unique winners may be less than \(N \) due to individuals with higher fitness winning multiple tournaments.

- **Selection Pressure:** Adjusting the tournament size \(T \) can control the selection pressure, with larger tournaments favoring the selection of the best individuals more strongly.

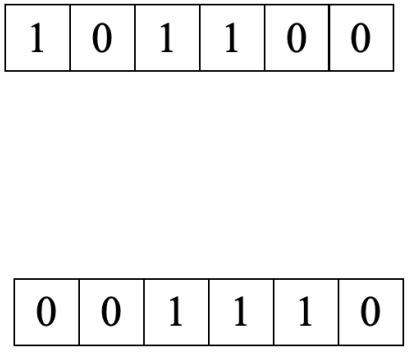
In summary, while the number of tournaments conducted is usually equal to the population size, the number of unique winners may vary depending on the fitness distribution and tournament size.

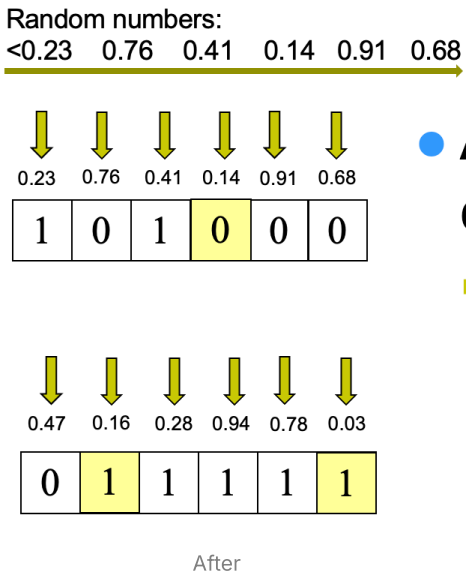
1PTX Crossover

$$(a \overset{0}{\vee} b) \wedge (\neg d \overset{1}{\vee} f) \wedge (\neg a \overset{2}{\vee} c) \wedge (b \overset{3}{\vee} \neg f) \wedge (\neg b \overset{4}{\vee} c) \wedge (c \overset{5}{\vee} e)$$



Mutation





Random numbers:

<0.23 0.76 0.41 0.14 0.91 0.68 0.47 0.19 0.28 0.94 0.78 0.03>

- A loop is performed on each individual.
  - If a random value $[0, 1]$ is $< p_m = 0.2$, then the allele value is flipped
  - otherwise kept the same

Replacement Strategies

▼ **Example: Transgenerational GA Replacement (no elitism)**

$$(a \overset{0}{\vee} b) \wedge (\neg d \overset{1}{\vee} f) \wedge (\neg a \overset{2}{\vee} c) \wedge (b \overset{3}{\vee} \neg f) \wedge (\neg b \overset{4}{\vee} c) \wedge (c \overset{5}{\vee} e)$$

Old Population 3 and 4 are selected as parents (two parents produce two offspring):

i	chrom. *abcdef*	unsat. clauses	fitness
1 :	110000	012345	3
2 :	010100	012345	3
3 :	001100	012345	2
4 :	101110	012345	1

Offspring:

i	chrom. *abcdef*	unsat. clauses	fitness
1 :	110000	012345	3
2 :	010100	012345	3
3 :	101000	012345	0
4 :	011111	012345	0

Technically the combined **temporary** population at this point looks like this **with offspring in blue**

i	chrom. *abcdef*	unsat. clauses	fitness
1 :	110000	012345	3
2 :	010100	012345	3
p1:	001100	012345	2
p2:	101110	012345	1
$o1$:	101000	012345	0
$o2$:	011111	012345	0

- Form the new generation **by copying offspring** onto the old population $(g = N = 4)$:

New Population we just copy the offspring over to where the parents would have been because of the replacement strategy:

Here we replace the old population with the new population by taking the children and putting them in and the other two parents (technically we do copy across n=4 things)

i	chrom. *abcdef*	unsat. clauses	fitn
1 :	110000	012345	3
2 :	010100	012345	3
3 :	101000	012345	0
4 :	011111	012345	0

▼ **Example: Transgenerational GA Replacement (with elitism)**

$$(a \overset{0}{\vee} b) \wedge (\neg d \overset{1}{\vee} f) \wedge (\neg a \overset{2}{\vee} c) \wedge (b \overset{3}{\vee} \neg f) \wedge (\neg b \overset{4}{\vee} c) \wedge (c \overset{5}{\vee} e)$$

Old Population:

i	chrom. *abcdef*	unsat. clauses	fitness
$1:$	110000	012345	3
$2:$	010100	012345	3
$3:$	001100	012345	2
$4:$	101110	012345	1

Offspring:

i	chrom. *abcdef*	unsat. clauses	fitness
$1:$	110000	012345	3
$2:$	010100	012345	3
$3:$	101000	012345	0
$4:$	011111	012345	0

Technically the combined **temporary** population at this point looks like this **with offspring in blue**

i	chrom. *abcdef*	unsat. clauses	fitn
$1:$	110000	012345	3
$2:$	010100	012345	3
p1:	001100	012345	2
p2:	101110	012345	1
$o1:$	101000	012345	0
$o2:$	011111	012345	0

- Form the new generation by selecting the best 4 individuals among both old population and offspring $(g = N = 4)$:
- Notice here now we replace the amount above 4 with 4 of the best from the temporary
- Here again we form the population with the best of the old and the children replacing 4 things
- sort (*N* + *aN*) [temp array] and choose the *N* best (elitism)

New Population:

i	chrom. *abcdef*	unsat. clauses	fitn
$1:$	001100	012345	2
$2:$	101110	012345	1
$3:$	101000	012345	0
$4:$	011111	012345	0

▼ **Example: Steady State GA Replacement (with elitism)**

$$(a \overset{0}{\vee} b) \wedge (\neg d \overset{1}{\vee} f) \wedge (\neg a \overset{2}{\vee} c) \wedge (b \overset{3}{\vee} \neg f) \wedge (\neg b \overset{4}{\vee} c) \wedge (c \overset{5}{\vee} e)$$

Old Population:

i	chrom. *abcdef*	unsat. clauses	fitness
$1:$	110000	012345	3
$2:$	010100	012345	3
$3:$	001100	012345	2
$4:$	101110	012345	1

Offspring:

- Method 2: replace the worst 2 of the population with 2 offspring $(g = N = 2)$:

New Population formed by taking the two offspring and replacing the next two worst of old population:

Here g=2 so we replace the worst performing of the old with the offspring

i	chrom. *abcdef*	unsat. clauses	fitn
$1:$	010100	012345	3
$2:$	101000	012345	0

Technically the combined **temporary** population at this point looks like this **with offspring in blue**

i	chrom. *abcdef*	unsat. clauses	fitness
$1:$	110000	012345	3
$2:$	010100	012345	3
p1:	001100	012345	2
p2:	101110	012345	1
$o1:$	010100	012345	3
$o2$	101000	012345	0

i	chrom. *abcdef*	unsat. clauses	fitn
$1:$	010100	012345	3
$2:$	101000	012345	0
$3:$	001100	012345	2
$4:$	101110	012345	1

TG (no elitism) → parents with offspring

TG (elitism) → replace the parents with offspring or parents

Termination Criteria

1. **Maximum Number of Generations**: Stop after 1000 generations have been evaluated, for example.
2. **Fitness Goal Achievement**: If the goal is to satisfy as many clauses as possible in a boolean formula, the algorithm could terminate if a solution is found that satisfies a predefined threshold, say 95% of the clauses.
3. **Population Convergence**: If all or most of the candidate solutions in the population represent the same or very similar sets of truth assignments to the variables, implying no new solutions are being generated, the algorithm might stop.
4. **Stagnation**: If the best fitness score hasn't improved for a certain number of generations, such as 100 consecutive generations, the algorithm would terminate.
5. **Combination of Conditions**: The algorithm might employ a combination of these criteria, such as stopping after 1000 generations or when the fitness improvement has stalled for 100 generations, whichever comes first.

- Example Scenario: If the algorithm has been running for 850 generations and the best solution's fitness score hasn't improved for the last 100 generations, the algorithm would terminate because it met the stagnation criterion.

▼ Another simpler example

- Let's take a simplified example of a genetic algorithm (GA) to optimize a problem. We'll use a GA to find the maximum value of a function $f(x) = x^2$, where x is a binary string of length 4 (so x represents a number between 0 and 15).

1. **Haploid structure**: We represent each solution as a single binary chromosome of length 4. For example, the number 6 would be represented as 0110.
2. **Initial Population**: Generate an initial population randomly.
    - Individual 1: 1101 (represents 13)
    - Individual 2: 1001 (represents 9)
    - Individual 3: 0110 (represents 6)
    - Individual 4: 0001 (represents 1)
3. **Calculate Fitness Values**: Evaluate each individual using the fitness function f(x).
    - Fitness(1101): 13^2 = 169
    - Fitness(1001): 9^2 = 81
    - Fitness(0110): 6^2 = 36
    - Fitness(0001): 1^2 = 1
4. **Selection**: Select individuals to be parents. Let's use a simple roulette wheel approach where the chance of being selected is proportional to fitness.
    - Likely parents: 1101 and 1001 (because they have higher fitness values)
5. **Crossover (Recombination)**: With a crossover probability (p_c), recombine pairs to create children.
    - Assuming p_c = 1 (always crossover) and we do a single point crossover between second and third bit, we might get:
    - Parent 1: 1101
    - Parent 2: 1001
    - Child 1: 1101 (crossed with Parent 2 after second bit) → 1101
    - Child 2: 1001 (crossed with Parent 1 after second bit) → 1001
6. **Mutation**: With a mutation probability (p_m), randomly flip some bits.
    - Assuming p_m = 0.25, let's say the second bit of Child 1 and the third bit of Child 2 are flipped:
    - Child 1 after mutation: 1001 (which still represents 9)
    - Child 2 after mutation: 1011 (which represents 11)
7. **New Population**: Replace the old population with the new children, depending on the strategy.
    - Let's replace the two least fit individuals:
    - New Individual 1: 1101 (old parent)
    - New Individual 2: 1001 (child, same as old parent)
    - New Individual 3: 1011 (mutated child)
    - New Individual 4: 1101 (old parent)
8. **Repeat the process**: Calculate fitness for the new population and continue the cycle.
    - Evaluate new individuals, select, crossover, and mutate.

9. **Termination**: Let's say our termination criteria is when we have run 10 generations or when we have a fitness value of 256 (the maximum for our function with 4-bit strings).
10. **Result**: After the termination criteria are met, the best individual in the last generation is taken as the solution.

▼ Another another example

Thank you for the clarification. Let’s break down the process step by step to address your question accurately.

Clarified Scenario

1. **Iteration 1:**
   - **Current Population:** Contains 5 chromosomes.
   - **Tournament Selection:**
     - Tournament size is set to 3.
     - The best out of 3 is selected as a parent for crossover.
   - **Crossover:**
     - If crossover happens with itself, a single offspring is generated.
   - **Replacement in Iteration 1:**
     - You are asking whether this single offspring temporarily replaces its parent for the rest of Iteration 1.
2. **Iteration 2:**
   - **Transgenerational (Elitism):**
     - The best individuals (elitism) are preserved.
     - The new generation is formed by selecting the best from both parents and offspring, replacing the worst individuals.

Detailed Steps and Explanation

Iteration 1

1. **Initial Population:**
   Assume the initial population has 5 chromosomes with their respective fitness values.

i	Chromosome	Fitness
1	A	5
2	B	3
3	C	8
4	D	2
5	E	6

2. **Tournament Selection:**

- Select 3 random individuals for the tournament.
- Assume the selected individuals are B, D, and E.
- The best individual (D) is selected as the parent for crossover.

3. **Crossover:**
   - Since crossover happens with itself (for this example), the offspring is essentially a copy of D.
   - **Parent (D):** Chromosome D with fitness 2.
   - **Offspring:** Chromosome D with fitness 2.
4. **Temporary Replacement in Iteration 1:**
   - The offspring temporarily replaces the parent in the population for the rest of Iteration 1.

   Temporary Population after Crossover:

i	Chromosome	Fitness
1	A	5
2	B	3
3	C	8
4	D (new)	2
5	E	6

Iteration 2

1. **Elitism and Selection:**
   - Combine the temporary population with the offspring generated during the iteration.
   - If elitism is applied, the best individuals are preserved.
2. **Forming the New Population:**
   - Select the best individuals among the parents and offspring to form the new generation.
   - For simplicity, assume we keep the best 4 individuals.

   Combined Population:

i	Chromosome	Fitness
1	A	5
2	B	3
3	C	8
4	D (new)	2
5	E	6
6	D (old)	2

Select the Best 4:

i	Chromosome	Fitness
1	D (new)	2

2	D (old)	2
3	B	3
4	E	6

Conclusion

- **Temporary Replacement:** In Iteration 1, the offspring temporarily replaces the parent in the population for the rest of the iteration.
- **Elitism and Transgenerational Replacement:** In Iteration 2, the best individuals are selected from both the temporary population and the offspring to form the new generation, replacing the worst individuals.

This approach ensures that the algorithm can explore new solutions while preserving high-quality individuals through elitism.

The number of offspring generated in each iteration of a genetic or memetic algorithm can be influenced by several factors, including the population size, the replacement strategy, and specific parameters set within the algorithm. Here are the key factors:

1. Population Size

Fixed Population Size:

- In many genetic and memetic algorithms, the population size is kept constant throughout the generations. If the population size is N, typically N offspring are generated to replace the current population.

2. Crossover and Mutation Operators

Crossover Rate:

- The crossover rate determines the proportion of the population that will undergo crossover to produce offspring. For example, a crossover rate of 0.8 means 80% of the population will be selected for crossover.

Number of Parents:

- In each iteration, parents are selected for crossover. If each crossover produces 2 offspring and you perform $N/2$ crossovers, you will generate N offspring, where N is the population size.

3. Replacement Strategy

Generational Replacement:

- In generational replacement, the entire population is replaced by offspring. Thus, the number of offspring is equal to the population size N.

Steady-State Replacement:

- In steady-state replacement, only a few individuals are replaced at a time. The number of offspring generated will depend on how many individuals are being replaced in each iteration.

Elitist Replacement:

- With elitism, a certain number of top individuals (elites) are preserved, and the rest are replaced by offspring. If the population size is N and k elites are preserved, then $N - k$ offspring are generated to maintain a constant population size.

4. Specific Algorithm Parameters

Tournament Size and Selection Pressure:

- Tournament selection size and selection pressure do not directly determine the number of offspring but influence which individuals are chosen to be parents, which indirectly affects the quality and diversity of offspring.

Example Scenarios:

Scenario 1: Fixed Population Size with Generational Replacement

- **Population Size (N):** 10
- **Crossover Rate:** 1.0 (100% of the population undergoes crossover)
- **Number of Offspring Generated:** 10

In this scenario, the entire population is replaced by offspring. So, if there are 10 individuals in the current population, 10 offspring will be generated to replace them.

Scenario 2: Elitist Replacement

- **Population Size (N):** 10
- **Number of Elites (k):** 2
- **Number of Offspring Generated:** $N - k = 10 - 2 = 8$

In this scenario, 2 elites are preserved from the current population, and 8 offspring are generated to replace the remaining individuals.

Scenario 3: Steady-State Replacement

- **Population Size (N):** 10
- **Individuals Replaced per Iteration:** 2
- **Number of Offspring Generated per Iteration:** 2

In this scenario, only 2 individuals are replaced in each iteration, so 2 offspring are generated.

Summary:

- The number of offspring generated in each iteration is primarily determined by the population size and the replacement strategy used.
- In fixed population size algorithms with generational replacement, the number of offspring equals the population size.
- In elitist replacement strategies, the number of offspring equals the population size minus the number of elites.

- In steady-state replacement, the number of offspring generated per iteration depends on how many individuals are being replaced in each iteration.

By adjusting these parameters and strategies, you can control the number of offspring produced in each iteration of the algorithm.

https://prod-files-secure.s3.us-west-2.amazonaws.com/259ac55a-7b50-42c6-b172-d031975b9e58/07ed4a21-1b98-4af0-a753-cac8b279c77a/5.3._QUIZ_Evolutionary_Algorithms_I_(Lecture_5)_Self-assessment_Activity__Attempt_review___moodle.Nottingham.pdf